



*Bachelor Thesis*

# Evaluating NER Methods in AI Voice Onboarding on AWS

PIETRO SCAPINI  
*Artificial Intelligence*

Academic Year 2024/2025



*Bachelor in Artificial Intelligence*



Pietro Scapini

# **Evaluating NER Methods in AI Voice Onboarding on AWS**

**Supervisors:** Antonino Nocera, University of Pavia  
Giuseppe Trotta, Hindeep S.r.l.



# Abstract

The design of an AI-based voice onboarding system represents an evolution towards more natural, inclusive and personalised registration processes. In the context of online dating, where authenticity and character traits are becoming increasingly important, this thesis introduces an innovative solution which demonstrates, thanks to collaboration with the start-up Hindeep S.r.l., how generating user profiles from voice input can offer a concrete and promising alternative to traditional text-based forms.

The developed prototype adopts a fully serverless architecture built on Amazon Web Services (AWS), integrating Amazon Transcribe for speech-to-text conversion and a Named Entity Recognition (NER) module for the structured extraction of information from spontaneous speech. The pipeline is orchestrated via AWS Lambda functions and API Gateway, while the frontend, developed in HTML, CSS, and JavaScript, enables voice recording, audio submission, and interactive visualization of the generated profile.

A comparative evaluation of different NER solutions was conducted to identify the most suitable approach for the dating domain. Three alternatives were analyzed: Amazon Comprehend as a managed service, the fine-tuning of a BERT model on a synthetic dataset generated with a Large Language Model (LLM), and a zero-shot strategy based on LLMs through prompt engineering. Although BERT showed good levels of accuracy in controlled conditions, the model's size and deployment costs limited its practical applicability. In contrast, the zero-shot LLM approach proved to be more flexible, robust in handling colloquial language, and better aligned with the requirements of a serverless architecture. The prototype developed for Hindeep S.r.l. demonstrates the feasibility of an innovative voice-based onboarding process, capable of simplifying the user experience while capturing personal information that is difficult to capture with traditional text-based forms. The integration of Artificial Intelligence techniques with a serverless architecture also confirms the possibility of developing advanced solutions while keeping costs and management complexity under control.

In conclusion, this thesis not only proposes a replicable technical model for applications based on structured extraction of voice data, but also highlights the main challenges faced and the solutions adopted, offering concrete insights for future developments such as conversational onboarding, prosodic and emotional analysis, or adaptation to domains beyond dating, including, for example, recruitment, customer support, and healthcare.

## Keywords

Voice onboarding, User profiles, Artificial Intelligence, Amazon Web Services (AWS), Serverless architecture, Speech-to-text, HTML, CSS, JavaScript, Named Entity Recognition (NER), BERT, Large Language Models (LLM)



# Contents

|          |                                                                                         |           |
|----------|-----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                     | <b>7</b>  |
| 1.1      | Project context: voice-based onboarding                                                 | 7         |
| 1.2      | The case study: Hindeep s.r.l. and Project Objectives                                   | 7         |
| 1.3      | Outline                                                                                 | 8         |
| <b>2</b> | <b>Analysis of the technologies used and system architecture</b>                        | <b>9</b>  |
| 2.1      | Organizational Approach: Adoption of Agile                                              | 9         |
| 2.2      | Overview of Services and Technologies Used                                              | 9         |
| 2.2.1    | Cloud computing and AWS                                                                 | 9         |
| 2.2.2    | Services used                                                                           | 10        |
| 2.2.3    | Onboarding interface: HTML, CSS and JavaScript                                          | 11        |
| 2.2.4    | Introduction to Named Entity Recognition (NER)                                          | 12        |
| 2.3      | System Architecture                                                                     | 14        |
| 2.3.1    | General scheme                                                                          | 14        |
| 2.3.2    | Operational flow                                                                        | 15        |
| 2.3.3    | Detailed role of each component                                                         | 16        |
| <b>3</b> | <b>System Development</b>                                                               | <b>17</b> |
| 3.1      | Frontend Development                                                                    | 17        |
| 3.1.1    | Supporting Technologies and Libraries                                                   | 17        |
| 3.1.2    | Audio Acquisition: Design of the Recording Module                                       | 18        |
| 3.1.3    | Sending Audio to the Backend: Encoding, Transmission, and Feedback                      | 19        |
| 3.1.4    | Profile Retrieval and Presentation: Interactive Visualization and Editing               | 20        |
| 3.2      | Serverless Backend Development                                                          | 21        |
| 3.2.1    | Upload and Transcription Function                                                       | 21        |
| 3.2.2    | Entity Extraction Function (NER)                                                        | 22        |
| 3.2.3    | Profile Retrieval Function                                                              | 23        |
| <b>4</b> | <b>Named Entity Recognition: A Comparison of Solutions</b>                              | <b>25</b> |
| 4.1      | Approach 1: Amazon Comprehend                                                           | 25        |
| 4.1.1    | Advantages (Pros)                                                                       | 25        |
| 4.1.2    | Limitations and Critical Issues (Cons)                                                  | 26        |
| 4.2      | Approach 2: Fine-Tuning a Custom BERT Model                                             | 26        |
| 4.2.1    | The Transformer Architecture: BERT                                                      | 27        |
| 4.2.2    | Dataset Creation via LLM                                                                | 28        |
| 4.2.3    | Exploratory Analysis and Model Training                                                 | 29        |
| 4.2.4    | Advantages and Limitations of the BERT Approach: from Fine-Tuning to Prompt Engineering | 30        |
| 4.3      | Approach 3: Zero-Shot NER with Large Language Models                                    | 31        |
| 4.3.1    | Advantages (Pros)                                                                       | 31        |
| 4.3.2    | Limitations and Critical Issues (Cons)                                                  | 32        |
| 4.4      | Critical Comparison: Fine-Tuned BERT vs. Zero-Shot LLM                                  | 32        |
| 4.4.1    | Testing on Real-World Cases                                                             | 33        |
| <b>5</b> | <b>Conclusions and future developments</b>                                              | <b>37</b> |
| 5.1      | Challenges Encountered and Adopted Solutions                                            | 37        |
| 5.2      | Improvements and Future Developments                                                    | 38        |
| 5.3      | Applications in Other Contexts                                                          | 38        |
| 5.4      | Final Considerations                                                                    | 39        |
|          | <b>References</b>                                                                       | <b>41</b> |



# Acknowledgments

First of all, I would like to thank all the people who have accompanied me along this course of study, sharing their time, energy and encouragement with me.

A special thanks goes to Hindeep S.r.l., who welcomed me and gave me the opportunity to gain experience, learn, and engage with new realities. In particular, I would like to thank Giuseppe for his constant guidance and ability to convey passion and knowledge, and Stefania and Donato for their availability and daily support.

I am grateful to my supervisor, Prof. Antonino Nocera, for following me with attention, availability, and expertise, and for providing valuable suggestions at every stage of this work.

Thanks to all my classmates and friends who accompanied and supported me along the way, making difficulties easier and achievements more meaningful. I am immensely grateful to my family, whose support and patience have allowed me to face each step with serenity.

Finally, I would like to thank the professors who, during my university career, made me passionate about the subject and shared stimulating and motivating knowledge; and a special thanks to all the teachers I met during my school career who gave me the opportunity to discover and cultivate my passion.



# Chapter 1

## Introduction

### 1.1 Project context: voice-based onboarding

In today's technological landscape, saturated with applications and web services, a user's first interaction with a new product has become a decisive moment. In this context, the user onboarding process plays a crucial role, as it is designed to guide new users from the moment of registration until the completion of the operation. Effective onboarding is the key to turning an initial approach into active engagement, thus drastically reducing the churn rate and laying the foundation for a long-term relationship with the user.

The growing competition for users' attention has pushed companies to move beyond traditional registration forms and guided tours, looking for more seamless, personalized, and intelligent solutions. Within this scenario of innovation, Artificial Intelligence has emerged as a radical transformative factor. The integration of AI into onboarding processes is revolutionizing the user experience, making it more efficient and deeply personalized.

AI technologies not only enable content personalization but are also redefining the very paradigm of interaction. The introduction of conversational and voice interfaces represents the new frontier, promising to make onboarding not only faster but also more natural and intuitive. This approach helps to break down the barriers between the user and technology, turning a potentially complex process into a simple dialogue.

The project I am going to develop is based on a fundamental advantage in terms of accessibility: a voice-based onboarding system. This opens the door to a wide range of users, including people with visual or motor disabilities, who might have difficulty interacting with traditional graphical interfaces. The ability to register and configure a service simply by speaking makes the technology more inclusive and universal.

### 1.2 The case study: Hindeep s.r.l. and Project Objectives

The case study for this project is provided by Hindeep S.r.l., an innovative startup operating in the online dating sector. In a highly competitive market, Hindeep aims to differentiate itself by offering a more authentic and profound user experience, moving away from superficial dynamics based exclusively on image. The central idea of the application is to foster connections based on personality and interests, traits considered more representative of an individual. In this scenario, the onboarding process is not just a formality, but rather the first and fundamental step in communicating the brand's values and collecting the key data required to define the user profile.

The primary goal of this project is the design and development of a functional prototype of an AI-based voice onboarding system specifically designed for Hindeep's dating app. The system is conceived to radically transform the registration process, allowing users to create their profile not by filling out textual forms, but simply by recording a voice message. The system aims to capture the essence of the user through their voice, then au-

tomatically extract the key information and populate their profile quickly and intuitively, also leveraging natural language processing techniques such as Named Entity Recognition (NER).

To implement this system, a completely serverless approach was adopted from an architectural point of view, orchestrated through Amazon Web Services (AWS) cloud services. This implementation methodology was chosen for its ability to build a complex, scalable, and cost-effective system, while avoiding the management of a traditional backend infrastructure. The prototype relies on a minimal frontend developed in HTML, CSS, and JavaScript, designed to asynchronously interact with cloud services and manage the entire onboarding flow.

## 1.3 Outline

The thesis examines the different aspects of the study in the following chapters:

**Chapter 2** analyses the technologies used and the system architecture, describing the Agile approach adopted for project management and the main AWS services employed (S3, Transcribe, Lambda, and API Gateway). It also presents the web onboarding interface and introduces the concepts of Named Entity Recognition, with references to datasets, encoding schemes, and evaluation metrics.

**Chapter 3** focuses on the development of the prototype, illustrating the frontend for voice recording, data submission, and interactive visualization of the generated profile. The backend section describes the Lambda functions that handle uploading and transcription, entity extraction, and profile retrieval, ensuring an asynchronous and fully serverless workflow.

**Chapter 4** presents a critical comparison between different solutions for Named Entity Recognition: the use of Amazon Comprehend, fine-tuning a BERT model on a synthetic dataset, and the zero-shot approach with LLMs through prompting. Advantages, limitations, and costs of each method are analysed, leading to the choice of the LLM-based approach for its flexibility and robustness in the application context.

**Chapter 5** presents the conclusions and future developments, describing the main challenges faced and the solutions adopted, the possible evolutions of the prototype, and its applications in other contexts, and then offers a final reflection on the value and potential of the developed system.

# Chapter 2

## Analysis of the technologies used and system architecture

### 2.1 Organizational Approach: Adoption of Agile

The Agile method was adopted for the management and development of the project, chosen for its ability to organize and adapt to the continuous changes of a work in progress. In a context like this, in which it was necessary to experiment with different tools and solutions and in which new requirements could progressively emerge, Agile proved to be essential to maintain flexibility and effectively deal with unforeseen events.

The practical application of this organizational method was realized in the subdivision of the entire workload into weekly sprints. Each sprint represented a short and self-contained development cycle, with clear and defined objectives. This temporal division allowed to have greater awareness of the work carried out and a better organization of future activities. Moreover, this iterative approach was very useful to manage the problems encountered along the way: at the end of each week, it was possible to analyse the difficulties that had emerged, such as unforeseen limits of a service or technical obstacles, and adapt the strategy for the next sprint by quickly seeking and testing new solutions.

The Trello project management tool was used to support this iterative work organisation. Through a virtual board, the activities were organized into columns representing their status (e.g. "To Do", "In Progress", "Completed"). This system offered a clear and constant view of the progress status of the project, allowing continuous tracking of progress and facilitating the planning of each new sprint. The representation offered by Trello was a key element to maintain focus on the objectives and ensure compliance with the timelines.

### 2.2 Overview of Services and Technologies Used

#### 2.2.1 Cloud computing and AWS

Amazon Web Services (AWS) is the world's most comprehensive and widely used cloud services platform and has been chosen as the basis for developing this project [1]. AWS offers a global infrastructure and a portfolio of over 200 services ranging from computational processing to storage, from machine learning to data analysis, allowing startups, large companies, and government agencies to develop sophisticated applications with great flexibility.

The decision to rely on AWS is part of the broader paradigm of Cloud Computing, a model that provides IT resources — such as servers, storage, databases, and a wide range of application services — via Internet on an on-demand basis. Unlike a traditional approach, which requires the purchase, configuration and maintenance of expensive physical infras-

structure, cloud computing allows computing power and services to be "rented" from a provider. This means that providers like AWS own and manage the hardware and network infrastructure, while the end user simply selects and uses the services needed for their applications, paying exclusively for the resources actually consumed according to the pay-as-you-go model.

The adoption of a cloud platform such as AWS brings several strategic advantages for companies, which can be summarized in the following points [1]:

- **Flexibility and cost control**  
Instead of incurring initial investments in hardware infrastructure (CapEx), the cloud allows you to adopt a variable spending model (OpEx), paying only for the resources you actually consume.
- **Economies of scale**  
Providers like AWS, by aggregating the demand of millions of customers, are able to offer advanced technologies at much lower costs than a company could sustain on its own.
- **Immediate scalability**  
It is no longer necessary to estimate computational capacity in advance: resources can be increased or reduced within minutes according to real needs, avoiding waste.
- **Speed and agility**  
In a cloud environment, new IT resources are available with just a few clicks, reducing provisioning times from weeks to minutes. This allows teams to experiment, prototype, and release new features in shorter time frames.
- **Focus on core business**  
By delegating infrastructure management (updates, maintenance, security) to AWS, companies can focus their efforts on developing applications and services that generate value and competitive differentiation.
- **Global distribution**  
With the cloud, it is possible to make an application available in multiple geographic regions quickly. This ensures users faster response times (low latency) and greater service reliability (resilience), since the system can continue operating even in case of failure in a single region.

Taking advantage of these benefits, the architecture of the project was developed as a serverless system, orchestrated through various AWS services that work together in a unified flow. For greater clarity, the main services used are described following the data path: from audio uploading to its processing up to the return of the result to the user.

### 2.2.2 Services used

#### Amazon S3 (Simple Storage Service)

Amazon S3 (Simple Storage Service) is an object storage service that offers developers and IT teams a highly scalable and resilient solution, designed to store and retrieve virtually unlimited amounts of data. Unlike block storage (such as a traditional hard disk), S3 treats files as complete "objects", to which are associated metadata and a unique identifier. Companies can store any type of object — such as HTML pages, source code files, images, and encrypted data — and access them through HTTP-based protocols.

Its main features are extremely high data durability, the ability to handle a large number of simultaneous users, and low costs. This makes it a convenient solution for a wide range of use cases, including backup and restore, disaster recovery, archiving, big data analytics, and content distribution [2].

**Amazon Transcribe**

Amazon Transcribe is an automatic speech recognition (ASR – Automatic Speech Recognition) service that uses advanced machine learning models to convert audio into text. The service can process audio files in different formats and languages, generating accurate transcriptions complete with punctuation, formatting, and optionally, timestamps for each word. Transcribe is designed to handle a wide variety of use cases, from transcribing contact center calls to subtitling multimedia content [2].

**AWS Lambda**

AWS Lambda is a serverless and event-driven compute service. The term "serverless" means that it is possible to run code without having to worry about provisioning or managing servers. AWS takes care of managing the underlying infrastructure, its scalability, security patches, and the operating system entirely. Lambda functions are "event-driven", meaning they are executed only in response to an event (trigger) such as uploading a file to S3, a request to an API endpoint, or a change in a database [2].

**AWS API Gateway**

Amazon API Gateway is a fully managed service that allows you to create, publish, monitor, and secure APIs in a simple and scalable way. It is the entry point ("front door") for applications to securely access backend data and functionalities. API Gateway automatically handles traffic management, access control, performance monitoring, and API versioning, ensuring the ability to seamlessly handle even hundreds of thousands of simultaneous requests [2].

**2.2.3 Onboarding interface: HTML, CSS and JavaScript**

The user interface (frontend) of the project, that is, the visual component with which the user interacts, was developed using the triad of standard web technologies: HTML, CSS, and JavaScript. This choice guarantees maximum compatibility and accessibility, constituting the foundation on which almost all modern web applications are based. Their success lies in the principle of separation of concerns, in which each language has a distinct and complementary role.

**HTML (HyperText Markup Language)**

HTML is the markup language that defines the logical and semantic structure of the contents of a web page. It does not determine the visual appearance, but organizes the elements hierarchically, using "tags" to identify titles, paragraphs, images, links, and forms. In this sense, it represents the skeleton of the document, assigning each piece of information a precise meaning and function, thus making it interpretable and coherent for browsers and navigation tools [3].

**CSS (Cascading Style Sheets)**

CSS is the language responsible for the presentation and style of the HTML document. Its task is to define the visual appearance of the elements: layout, colors, typography, spacing, and animations. Working in synergy with HTML, CSS allows content to be completely separated from its aesthetic form. This separation makes the code cleaner, more flexible, and allows the entire design of an application to be modified without altering its HTML structure [3].

**JavaScript**

JavaScript is the programming language that gives interactivity and dynamism to web pages. If HTML is the structure and CSS the style, JavaScript is the engine that manages the behaviour of the application. It operates on the client side and allows manipulation of the structure (DOM) and style of the page in real time, reacting to user events (such as clicks and keyboard input), validating data, and above all communicating asynchronously with

a server to send and receive information without reloading the page. It is the technology that transforms a static document into a complex and responsive web application [3].

### 2.2.4 Introduction to Named Entity Recognition (NER)

Named Entity Recognition (NER) is a central area of Information Extraction (IE) within Natural Language Processing (NLP). Its goal is to automatically identify and classify the so-called named entities, that is, portions of text that refer to elements of the real world. In the context of this project, NER plays a fundamental role because it enables the extraction of relevant entities in an automatic and consistent manner.

#### Reference datasets and limitations for the project

There are numerous standard datasets in the literature for training and evaluating NER models, as shown in the table below.

| Corpus         | Year      | Text Source                    | #Tags | URL                                                                                                                                                                                                                   |
|----------------|-----------|--------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MUC-6          | 1995      | Wall Street Journal            | 7     | <a href="https://catalog.ldc.upenn.edu/LDC2003T13">https://catalog.ldc.upenn.edu/LDC2003T13</a>                                                                                                                       |
| MUC-6 Plus     | 1995      | Additional news to MUC-6       | 7     | <a href="https://catalog.ldc.upenn.edu/LDC96T10">https://catalog.ldc.upenn.edu/LDC96T10</a>                                                                                                                           |
| MUC-7          | 1997      | New York Times news            | 7     | <a href="https://catalog.ldc.upenn.edu/LDC2001T02">https://catalog.ldc.upenn.edu/LDC2001T02</a>                                                                                                                       |
| CoNLL03        | 2003      | Reuters news                   | 4     | <a href="https://www.clips.uantwerpen.be/conll2003/ner/">https://www.clips.uantwerpen.be/conll2003/ner/</a>                                                                                                           |
| ACE            | 2000–2008 | Transcripts, news              | 7     | <a href="https://www.ldc.upenn.edu/collaborations/past-projects/ace">https://www.ldc.upenn.edu/collaborations/past-projects/ace</a>                                                                                   |
| OntoNotes      | 2007–2012 | Magazine, news, web, etc.      | 18    | <a href="https://catalog.ldc.upenn.edu/LDC2013T19">https://catalog.ldc.upenn.edu/LDC2013T19</a>                                                                                                                       |
| W-NUT          | 2015–2018 | User-generated text            | 6/10  | <a href="http://noisy-text.github.io">http://noisy-text.github.io</a>                                                                                                                                                 |
| BBN            | 2005      | Wall Street Journal            | 64    | <a href="https://catalog.ldc.upenn.edu/LDC2005T33">https://catalog.ldc.upenn.edu/LDC2005T33</a>                                                                                                                       |
| WikiGold       | 2009      | Wikipedia                      | 4     | <a href="https://figshare.com/articles/Learning_multilingual_named_entity_recognition_from_Wikipedia/5462500">https://figshare.com/articles/Learning_multilingual_named_entity_recognition_from_Wikipedia/5462500</a> |
| WiNER          | 2012      | Wikipedia                      | 4     | <a href="http://rali.iro.umontreal.ca/rali/en/winer-wikipedia-for-ner">http://rali.iro.umontreal.ca/rali/en/winer-wikipedia-for-ner</a>                                                                               |
| WikiFiger      | 2012      | Wikipedia                      | 112   | <a href="https://github.com/xiaoling/figer">https://github.com/xiaoling/figer</a>                                                                                                                                     |
| HYENA          | 2012      | Wikipedia                      | 505   | <a href="https://www.mpi-inf.mpg.de/departments/databases-and-information-systems/research/yago-naga/hyena/">https://www.mpi-inf.mpg.de/departments/databases-and-information-systems/research/yago-naga/hyena/</a>   |
| N <sup>3</sup> | 2014      | News                           | 3     | <a href="http://aksw.org/Projects/N3NER/NEDNIF.html">http://aksw.org/Projects/N3NER/NEDNIF.html</a>                                                                                                                   |
| Gillick        | 2016      | Magazine, news, web, etc.      | 89    | <a href="https://arxiv.org/e-print/1412.1820v2">https://arxiv.org/e-print/1412.1820v2</a>                                                                                                                             |
| FG-NER         | 2018      | Various                        | 200   | <a href="https://fgner.alt.ai/">https://fgner.alt.ai/</a>                                                                                                                                                             |
| NNE            | 2019      | Newswire                       | 114   | <a href="https://github.com/nickyringland/nested_named_entities">https://github.com/nickyringland/nested_named_entities</a>                                                                                           |
| GENIA          | 2004      | Biology and clinical text      | 36    | <a href="http://www.geniaproject.org/home">http://www.geniaproject.org/home</a>                                                                                                                                       |
| GENETAG        | 2005      | MEDLINE                        | 36    | <a href="https://sourceforge.net/projects/bioc/files/">https://sourceforge.net/projects/bioc/files/</a>                                                                                                               |
| FSU-PRGE       | 2010      | PubMed and MEDLINE             | 5     | <a href="http://julielab.de/Resources/FSU_PRGE.html">http://julielab.de/Resources/FSU_PRGE.html</a>                                                                                                                   |
| NCBI-Disease   | 2014      | PubMed                         | 24    | <a href="https://www.ncbi.nlm.nih.gov/CBBresearch/Dogan/DISEASE/">https://www.ncbi.nlm.nih.gov/CBBresearch/Dogan/DISEASE/</a>                                                                                         |
| BC5CDR         | 2015      | PubMed                         | 3     | <a href="http://bioc.sourceforge.net/">http://bioc.sourceforge.net/</a>                                                                                                                                               |
| DFKI           | 2018      | Business news and social media | 7     | <a href="https://dfki-lt-re-group.bitbucket.io/product-corpus/">https://dfki-lt-re-group.bitbucket.io/product-corpus/</a>                                                                                             |

Table 2.1: Overview of NER corpora [4].

These corpora, such as CoNLL03 (based on Reuters news) or OntoNotes, have been crucial for the development of the field, but they focus on a set of generic entities (persons, locations, organizations) extracted from formal texts.

For the purposes of this project, an analysis of these datasets revealed that none of them is suitable for handling colloquial language and the specific entities of a dating app profile, as they lack fundamental categories such as HOBBY, INTERESSE\_PARTNER or RANGE\_ETA. This gap made it necessary to evaluate customized approaches, as it will be discussed in Chapter 4.

### Operating principles and data construction

NER models, like most machine learning systems, are based on learning from data. To train them, it is necessary to have an annotated text corpus, that is, a collection of sen-

tences in which entities have been manually identified and tagged. This process, although time and resource consuming, is essential to provide the model with examples from which to learn the linguistic and contextual patterns that characterize entities.

A classic example is shown in the following figure, where some words are highlighted and classified into categories such as PER (person), ORG (organization), and LOC (location).

|                                                                                                                                                                                                                                                                                                                                                                                                                                                     |            |                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------------------------------|
| <p>CRICKET – <b>MILLNS</b> SIGNS FOR <b>BOLAND</b></p> <p><b>CAPE TOWN</b> 1996–08–22</p> <p>South African provincial side <b>Boland</b> said on Thursday they had signed <b>Leicestershire</b> fast bowler <b>David Millns</b> on a one year contract. <b>Millns</b>, who toured <b>Australia</b> with <b>England</b> A in 1992, replaces former <b>England</b> all-rounder <b>Phillip DeFreitas</b> as <b>Boland's</b> overseas professional.</p> | Labels:    | Examples:                         |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <b>PER</b> | David Millns<br>Philip DeFreitas  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <b>ORG</b> | Boland                            |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <b>LOC</b> | Cape Town<br>Australia<br>England |

**Figure 2.1:** Example of annotated text for NER task. Words constituting entities are associated with a predefined label (e.g., PER, ORG, LOC) [5].

It is precisely thanks to thousands of such annotations that a model learns to automatically recognize new entities in different texts.

In the context of this project, in addition to the traditional approach, the possibility of generating an annotated dataset using a Large Language Model (LLM), to be used later to train a recognition model, was also considered. This option, with its advantages and limitations, will be discussed in more detail in Chapter 4.

### Encoding schemes for entity recognition

To enable the model to handle entities composed of multiple words (e.g., "Mario Rossi"), it is not sufficient to label individual words; a coding scheme that defines their boundaries is also required.

#### BIO SCHEME (BEGINNING, INSIDE, OUTSIDE)

This is the most common and widely used scheme, also known as IOB [6]. Each word (token) in the sentence is assigned a label:

- B-[TYPE]: the word is the Beginning of an entity (e.g., B-PER).
- I-[TYPE]: the word is Inside an entity (e.g., I-PER).
- O: the word is Outside any entity.

For example: Mario [B-PER] Rossi [I-PER] works [O] for [O] Google [B-ORG].

#### ALTERNATIVE SCHEMES (E.G., BILOU/IOBES)

To provide the model with even more precise information, more detailed schemes exist [6]. The BILOU scheme (Beginning, Inside, Last, Outside, Unit) or IOBES scheme (Inside, Outside, Beginning, End, Single) adds labels to specify:

- L-[TYPE] or E-[TYPE]: the word is the Last/End of a multi-word entity.
- U-[TYPE] or S-[TYPE]: the word is a Single-word entity (Unit/Single).

With this scheme, the previous example becomes: Mario [B-PER] Rossi [L-PER] works [O] for [O] Google [U-ORG]. This additional detail can help the model learn entity boundaries better, leading to potential performance improvements.

## Evaluation metrics

Evaluating an NER system is more complex than simply calculating accuracy. A model could achieve very high accuracy by labelling all words as O (Outside) — an apparently correct choice in texts where there are few entities — but in reality, this would result in a completely useless system.

For this reason, more specific metrics are adopted, borrowed from the field of Information Retrieval, which take into account the effective recognition of entities [6]:

- **Precision:** indicates the correctness of the entities found by the model. It answers the question: "Among all the entities identified, how many were actually correct?"  

$$\text{Precision} = \frac{TP}{TP+FP}$$
 where TP (True Positives) are correctly identified entities, and FP (False Positives) are incorrectly identified ones.
- **Recall:** measures the model's ability to identify all entities present in the text. It answers the question: "Among all existing entities, how many did the model find?"  

$$\text{Recall} = \frac{TP}{TP+FN}$$
 where FN (False Negatives) are entities present but not recognized.
- **F1-Score:** the harmonic mean of Precision and Recall, representing the main metric for evaluating an NER model, as it balances the system's ability to be both precise and comprehensive. 
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

## 2.3 System Architecture

### 2.3.1 General scheme

This section presents the system architecture, with the main technological components adopted and their logical interconnections. The entire system has been designed following serverless and event-driven paradigms [1], in order to ensure scalability, resilience, and low costs. Each service operates independently, interacting through events and without rigid dependencies on others.

Figure 2.2 shows the complete data flow: from the acquisition of the voice input by the client to the storage and visualization of the processed user profile.

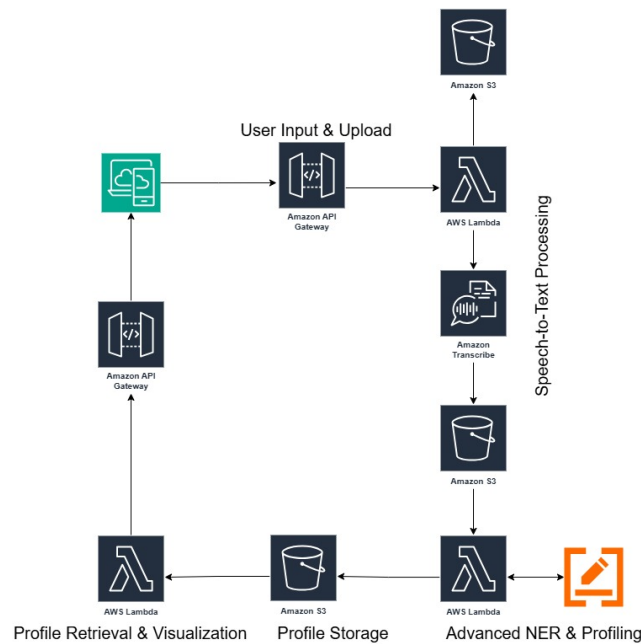


Figure 2.2: Architectural scheme of the voice onboarding system based on AWS services.

As can be seen from the scheme, the architecture includes a frontend responsible for user interaction and a fully serverless backend hosted on Amazon Web Services. The frontend does not interact directly with individual services, but communicates solely through Amazon API Gateway, which serves as a secure and managed entry point.

The core of the process is asynchronous: uploading the audio file to Amazon S3 triggers a chain of events that activates various AWS Lambda functions. These orchestrate the entire workflow, invoking specialized services such as Amazon Transcribe for speech-to-text conversion and a Named Entity Recognition (NER) model for extracting useful profile information. After several experiments, the final solution for NER was based on the use of a Large Language Model (LLM) (DeepSeek), capable of more effectively handling colloquial language and the specific entities required by the context. The reasons behind this choice will be further discussed in Chapter 4.

S3 buckets act as intermediate datastores, collecting audio files, transcriptions, and final profiles, which are then made available to the frontend for visualization and potential modification.

### 2.3.2 Operational flow

The architecture operates according to a well-defined flow of data and events, which orchestrates various cloud services asynchronously. The entire process, from voice recording to profile display, consists of the following operational phases:

1. **Pipeline Loading and Launch (Frontend → API Gateway → Lambda\_1 → S3\_1 & Transcribe):**  
The interaction begins on the frontend, where the user records their voice message. Once completed, the audio file is encoded in Base64 format and sent via a POST request to an Amazon API Gateway endpoint. The API Gateway triggers the first Lambda function, which acts as the initial orchestrator of the process. This function performs two critical tasks in sequence: first, it decodes the audio data and saves it as an object in an Amazon S3 bucket; immediately after, it initiates a transcription process on Amazon Transcribe, instructing the service to analyse the newly uploaded file.
2. **Asynchronous Transcription and NER Activation (Transcribe → S3\_2 → Lambda\_2):**  
Amazon Transcribe operates asynchronously to convert audio into text. Once the analysis is complete, the service saves the result (a JSON file containing the transcription) in a second S3 bucket.  
The storage of this file generates an event `s3:ObjectCreated:*` that triggers the second Lambda function, which is crucial to the operation of the system.
3. **Entity Extraction via LLM (Lambda\_2 → DeepSeek → S3\_3):**  
The second Lambda function reads the transcribed text from the JSON file. Instead of using a traditional NER model (see Chapter 4 for a discussion of the motivations behind this choice), it implements a prompt engineering technique: a detailed prompt instructs an external Large Language Model (DeepSeek) on how to extract and structure the entities of interest (name, age, hobbies, etc.). The function makes an API call to this model, receives the response in JSON format, and saves the structured user profile in a third S3 bucket dedicated to final data.
4. **Profile Request and Retrieval (Frontend → API Gateway → Lambda\_3):**  
Meanwhile, the user interface waits for the processing to complete. It then makes a GET request to a second Amazon API Gateway endpoint to retrieve the profile. The API Gateway forwards the request to a third Lambda function, whose responsibility is to query the bucket containing the final profiles.

#### 5. Data Visualization (Lambda\_3 → Frontend):

The third Lambda function does not retrieve a specific file but implements logic to identify and read the most recent object present in the output bucket, assuming it corresponds to the latest processing result. This JSON object is then returned to the client through the API Gateway. Finally, the frontend JavaScript code receives the data and uses it to dynamically populate the form fields, displaying the complete profile to the user.

As can be seen, the overall functioning of the system relies on the coordinated interaction of different cloud services, each with a precise role within the processing flow.

### 2.3.3 Detailed role of each component

To complete the description, the individual components of the architecture are detailed below, with their respective responsibilities and contributions to the overall process:

- **Frontend (Client Application):** Acts as the user interface (UI) and the sole point of interaction with the user. Its responsibilities include capturing voice input via browser APIs, communicate asynchronously with the backend through endpoints exposed by the API Gateway, and finally present the processed data within an editable profile.
- **Amazon S3 (Simple Storage Service):** Serves as the object datastore for the entire process. Its role is threefold: it acts as a repository for raw audio files uploaded by the user, as the destination for transcription files generated by Amazon Transcribe, and finally as the archive for the final JSON profiles processed by the NER logic. It is also the main event generator for triggering Lambda functions.
- **AWS Lambda:** This is the computational brain and orchestrator of the serverless backend. The Lambda functions execute the entire system logic: they handle incoming requests from the API Gateway, invoke external services (such as Transcribe and the NER model), process intermediate data, and read/write objects from S3 buckets.
- **Amazon Transcribe:** Operates as a specialised service for speech-to-text conversion. Its sole role within the architecture is to receive the location of an audio file and return a structured textual transcription in JSON format, completely abstracting away the complexity of automatic speech recognition.
- **Amazon API Gateway:** Acts as a secure and managed front door for the backend. Its responsibility is to expose public HTTP endpoints, handle incoming client requests, forward them to the appropriate Lambda functions for processing, and return responses to the frontend, managing aspects such as traffic and security.
- **DeepSeek (External Large Language Model):** Represents the artificial intelligence component for entity extraction. Its role is to receive a text and a prompt with specific instructions and return a structured JSON output containing the identified entities, functioning as an "NER-as-a-service" component for the second Lambda function.

The architectural description has thus highlighted how each component contributes to the overall process of voice onboarding. The next chapter will analyse the implementation aspects, showing in detail the development of the system on both the frontend and serverless backend sides.

# Chapter 3

## System Development

After defining the architecture in the previous chapter, this chapter focuses on the implementation aspects of the project. The code and the specific logics used to build both the frontend, i.e., the interface with which the user interacts, and the serverless backend that orchestrates the entire processing workflow will be analysed.

### 3.1 Frontend Development

The frontend represents the interface through which the user directly interacts with the system. It was developed as a minimal and functional web application; the design aimed at usability fluidity, accessibility, and the ability to integrate transparently with the services of the serverless backend.

The interface has an aesthetic character that aims to communicate the expressed objectives, capable of guiding the user through a two-step process:

1. voice recording,
2. subsequent visualization with the possibility of modifying the generated profile.

The development of the project followed some fundamental guiding principles:

- **Ease of use and intuitiveness:** provide a simple, linear, and intuitive interface, with recognizable buttons and descriptive texts that guide the user step by step.
- **Constant feedback:** set up visual and textual mechanisms that inform the user about the progress of the process, with a perception of limited waiting time.
- **Accessibility:** ensure application usability for a wide range of users, adopting adequate colour contrasts, textual annotations, and explanatory placeholders in the fields.
- **Integration with the backend:** facilitate fast and secure communication with the AWS architecture through the endpoints exposed by Amazon API Gateway.

#### 3.1.1 Supporting Technologies and Libraries

As anticipated in Chapter 2.2.3, the interface was built on the standard web triad: HTML, CSS, and JavaScript, without adopting complex frameworks in order to keep the prototype light and focused on the main functionalities. In addition to these basic languages, some key technologies were used:

- **Native Browser APIs.** The development relied on APIs provided directly by modern browsers, in particular:
  - **MediaRecorder API:** for capturing audio from the user's microphone. This interface allows easy recording of audio streams and their manipulation as data objects [7].
  - **Fetch API:** for all asynchronous communication with the serverless backend. It is the current standard for making HTTP requests, essential for sending audio and receiving profile data [8].
  - **FileReader API:** used to convert binary audio data into a textual format (Base64), necessary for sending via JSON [9].
- **External Libraries.** The interface was integrated with a JavaScript library:
  - **Chart.js:** an open-source library for creating charts. In the project, it was used to implement the radar chart that intuitively visualizes the user's personality traits, making the profile more dynamic and interactive [10].

### 3.1.2 Audio Acquisition: Design of the Recording Module

The first functional component of the application is the recording interface, which captures the user's voice input. It was designed to be intuitive and straightforward, as it manages the entire lifecycle of the audio recording. The logical steps of which it is composed were implemented in JavaScript as follows:

1. **Start of Recording:** When the user clicks on the "Start Recording" button, the function `navigator.mediaDevices.getUserMedia()` is invoked [7], requesting permission to access the microphone. Once consent is granted, a new instance of `MediaRecorder` is created.
2. **Audio Data Capture:** `MediaRecorder` begins capturing the audio stream. The data is collected in "chunks" (fragments) and stored in an array.
3. **Finalization of Recording:** When the user clicks "Stop Recording" the method `mediaRecorder.stop()` is triggered. This event finalizes the collection process and merges all the audio fragments into a single `Blob` object, a data type representing raw files. This `Blob` is then used to generate a temporary URL through `URL.createObjectURL()`, allowing the user to replay their recording in the `<audio>` element before submitting it.
4. **Interface Management:** During this process, the interface is dynamically updated: the recording button is hidden, and the "Retry" and "Submit" buttons appear, providing the user with clear control over the next step.

The following code snippet illustrates the implementation of this logic.

```

1 // Add a click listener to the record button
2 recordButton.addEventListener('click', async () => {
3   // If not currently recording, start the process
4   if (!isRecording) {
5     try {
6       // 1. Request access to the microphone
7       const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
8       mediaRecorder = new MediaRecorder(stream);
9       audioChunks = []; // Reset audio chunks array
10
11      // 2. Collect audio data as it becomes available
12      mediaRecorder.addEventListener('dataavailable', e => audioChunks.push(e.data));
13
14      // 3. When recording stops, assemble the audio data
15      mediaRecorder.addEventListener('stop', () => {

```

```

16     audioBlob = new Blob(audioChunks, { type: 'audio/webm' });
17     audioPlayer.src = URL.createObjectURL(audioBlob);
18
19     // Update the UI to show the player and action buttons
20     recordButton.style.display = 'none';
21     buttonGroup.style.display = 'flex';
22   });
23
24   // Start recording and update the state
25   mediaRecorder.start();
26   isRecording = true;
27   recordButton.textContent = 'Ferma Registrazione';
28 } catch (err) {
29   console.error("Error accessing the microphone:", err);
30   alert("Microphone permission denied.");
31 }
32 } else {
33   // If already recording, stop it
34   mediaRecorder.stop();
35   isRecording = false;
36   recordButton.textContent = 'Inizia Registrazione';
37 }
38 });

```

### 3.1.3 Sending Audio to the Backend: Encoding, Transmission, and Feedback

Once the audio acquisition is completed, the next step is the transmission of the data to the backend. This phase is technically crucial, as it requires preparing the binary audio file for transport over the HTTP protocol and managing asynchronous communication with the API. The process is structured into the following steps:

1. **Base64 Conversion:** The audio file is binary data and cannot be directly included in a JSON request. To overcome this limitation, the `FileReader` API is used with the `readAsDataURL()` method [9], which reads the contents of the `Blob` and converts it into a Base64-encoded string, i.e., a textual representation of binary data.
2. **Asynchronous API Call:** The Base64 string is placed into a JSON object and sent through a POST request to the Amazon API Gateway endpoint using the `fetch` API [8].
3. **User Feedback:** While waiting for the backend to process the request, the interface displays a loading animation along with a series of rotating messages (handled by the functions `startLoadingMessages` and `changeMessage`). This feature enhances the user experience by providing visual feedback that the system is actively working. Once the transmission is completed, the user can proceed to the second phase: profile visualization.

The implementation of this flow is illustrated in the code below.

```

1 // Add a click listener to the send button
2 sendButton.addEventListener('click', () => {
3   // Show the loading animation and disable the button
4   loadingDiv.style.display = 'flex';
5   sendButton.disabled = true;
6   startLoadingMessages();
7
8   const reader = new FileReader();
9
10  // 1. When the reader finishes loading the file
11  reader.onloadend = () => {
12    // 2. Convert the result to a Base64 string (removing the Data URL header)
13    const base64 = reader.result.split(',')[1];
14
15    // 3. Make the POST request to the API Gateway endpoint
16    fetch("https://[ID_API_GATEWAY].execute-api.[REGIONE].amazonaws.com/default/", {
17      method: "POST",
18      headers: { "Content-Type": "application/json" },
19      body: JSON.stringify({ audio: base64 }) // Send the audio in the JSON body
20    });

```

```

21 .then(res => console.log("â API response received", res))
22 .catch(e => console.error("â API Error", e));
23 };
24
25 // Start reading the audio Blob as a Data URL
26 reader.readAsDataURL(audioBlob);
27 });

```

### 3.1.4 Profile Retrieval and Presentation: Interactive Visualization and Editing

The final stage of the onboarding flow consists of displaying the user profile. This phase has the dual purpose of retrieving the data processed by the backend and presenting it to the user in an interactive interface that allows viewing and, if necessary, editing. The logic implemented to achieve this goal is divided into the following operations:

1. **Data Retrieval and Integration:** Once the backend has completed the processing and the profile is available, the application performs a `fetch` GET request to the dedicated endpoint [8]. This request downloads the JSON containing the information extracted from the voice recording, which will form the user profile. The JavaScript code then processes the response and maps the received values to the corresponding fields of the HTML form (`<input>`, `<select>`, `<textarea>`). If there is missing information, the fields remain empty, leaving the user free to complete them manually.
2. **Personality Traits Visualization:** The data related to personality traits is used to update the `Chart.js` radar chart [10], providing an immediate visual representation of the extracted personality profile.
3. **User Interaction:** At this stage, the profile is visible and editable, allowing the user to correct potential inaccuracies, modify values, or complete missing fields. By moving the personality sliders, the radar chart updates in real time thanks to the `oninput` event that calls the `updateRadarChart()` function, offering a complete and interactive experience.

The following code illustrates the logic behind retrieving and displaying the profile data.

```

1 // Add a click listener to the 'Load Profile' button
2 document.getElementById('loadProfileBtn').addEventListener('click', async () => {
3   // ... (UI management for loader and error messages) ...
4   try {
5     // 1. Call the profile endpoint to get the data
6     const response = await fetch(apiUrl);
7     if (!response.ok) {
8       throw new Error(`HTTP error! Status: ${response.status}`);
9     }
10    const data = await response.json();
11
12    // Handle the wrapped response from API Gateway
13    const profile = data.statusCode ? JSON.parse(data.body) : data;
14
15    // 2. Populate the form fields with the received data
16    document.getElementById('nome').value = profile.NOME || "";
17    document.getElementById('eta').value = profile.ETA || "";
18    document.getElementById('citta').value = profile.CITTA || "";
19    // ... (and so on for all other fields) ...
20
21    // 3. Update the radar chart data
22    if (Array.isArray(profile.LIVELLI_PERS)) {
23      radarChart.data.datasets[0].data = profile.LIVELLI_PERS;
24      radarChart.update();
25    }
26  } catch (error) {
27    // ... (error handling) ...
28  }
29 });

```

## 3.2 Serverless Backend Development

The frontend represents the point of contact with the user, while the serverless backend is the engine that executes all data processing logic invisibly to the end user. As mentioned in Chapter 2, the architecture is based entirely on services managed by AWS, eliminating the need to manage physical or virtual servers and allowing us to focus exclusively on application logic.

The backend development materialized in the creation of three distinct AWS Lambda functions, each with a specific trigger and a clearly defined responsibility within the asynchronous processing flow. These functions, developed in Python and orchestrated through events generated by Amazon S3 and API Gateway requests, constitute the backbone of the entire system. The implementation of each function is analysed below.

### 3.2.1 Upload and Transcription Function

The first Lambda function acts as the entry point for the backend. It is designed to receive the raw audio data from the frontend and initiate the processing pipeline.

- **Trigger:** The function is activated by Amazon API Gateway, which receives a POST request from the frontend.
- **Responsibilities:** Its dual responsibility is (1) to securely store the received audio file and (2) to start the transcription process.

The implementation begins with the decoding of the JSON payload received from API Gateway. The Base64 string containing the audio is converted back into its original binary format [11], as shown in the following code snippet.

```
1 # Import necessary libraries
2 import boto3
3 import base64
4 import uuid
5 import json
6
7 s3 = boto3.client('s3')
8
9 # Decode the request body and the Base64 audio
10 body = json.loads(event['body'])
11 audio_base64 = body['audio']
12 audio_data = base64.b64decode(audio_base64)
```

Subsequently, to ensure that each audio file has a unique name and to prevent accidental overwrites, a universally unique identifier (UUID) is generated through Python's uuid library [12]. This UUID is used as the filename. Using the Boto3 client for S3 [13], the file is then uploaded to the designated bucket.

```
1 AUDIO_BUCKET = 'file-audio-onboarding'
2
3 # Generate a unique filename using UUID
4 filename = f"{uuid.uuid4()}.webm"
5
6 # Save the audio data to the specified S3 bucket
7 s3.put_object(
8     Bucket=AUDIO_BUCKET,
9     Key=filename,
10     Body=audio_data,
11     ContentType='audio/webm'
12 )
```

Once the file has been saved, the function starts the transcription process on Amazon Transcribe. This operation, also managed via Boto3, is asynchronous: the function does not wait for the transcription to be completed, but simply starts it.

```
1 transcribe = boto3.client('transcribe')
2 TRANSCRIBE_OUTPUT_BUCKET = 'file-transcribe-onboarding'
3
4 # Create the S3 URI for the newly uploaded audio file
```

```

5 s3_uri = f"s3://{AUDIO_BUCKET}/{filename}"
6 # Use the unique AWS request ID as the transcription job name
7 job_name = context.aws_request_id
8
9 # Start the asynchronous transcription job on Amazon Transcribe
10 transcribe.start_transcription_job(
11     TranscriptionJobName=job_name,
12     Media={"MediaFileUri": s3_uri},
13     MediaFormat="webm",
14     OutputBucketName=TRANSCRIBE_OUTPUT_BUCKET,
15     LanguageCode="it-IT"
16 )

```

The function concludes by returning a success response (`statusCode: 200`) to the frontend, indicating that the process has been successfully initiated.

### 3.2.2 Entity Extraction Function (NER)

The second Lambda function is the component responsible for executing Named Entity Recognition (NER), the central process of extracting entities from the transcribed text.

- **Trigger:** The function is activated by an Amazon S3 event, configured to react to the upload of a new JSON file in the Transcribe output bucket.
- **Responsibilities:** Its task is to read the transcribed text, perform NER, and save the structured profile.

Upon activation, the function receives an event object containing information about the file that triggered it. From this object, the bucket name and the key (filename) of the transcription file are extracted.

```

1 # Extract bucket and file information from the S3 trigger event
2 record = event["Records"][0]
3 bucket_name = record["s3"]["bucket"]["name"]
4 file_name = record["s3"]["object"]["key"]
5
6 # Download and read the transcription JSON file from S3
7 response = s3.get_object(Bucket=bucket_name, Key=file_name)
8 content = response['Body'].read().decode('utf-8')
9 data = json.loads(content)
10
11 # Extract the transcribed text from the JSON structure
12 testo = data["results"]["transcripts"][0]["transcript"]

```

Once the text is retrieved, the function implements a prompt engineering technique. Instead of using a traditional NER model, it constructs a detailed prompt that instructs a Large Language Model on how to extract the entities and, most importantly, how to format the output into a structured JSON [14].

The model call is made via an endpoint provided by the OpenRouter platform [15], which aggregates various LLM APIs; for this project, the DeepSeek model was used.

```

1 # Build the detailed prompt for the Large Language Model
2 prompt = f"""Estrai le seguenti entità dal testo: NOME, ETA (...), HOBBY (...).
3
4 Testo:
5 "{testo}"
6
7 Rispondi **solo** con il contenuto JSON, senza usare blocchi Markdown...
8 {{
9     "NOME": "...",
10    "ETA": "...",
11    ...
12 }}"""

```

This prompt is then sent via a POST request to the model's endpoint using Python's requests library [16]. The model's response is parsed and stored in the final S3 bucket, as illustrated in the code below.

```

1 # ... (code to prepare payload and headers for the API call) ...
2
3 # Execute the API call to the LLM endpoint
4 api_response = requests.post(
5     url=DEEPSEEK_ENDPOINT,
6     headers=headers,
7     data=json.dumps(payload),
8     timeout=20
9 )
10 api_response.raise_for_status() # Raise an exception for bad status codes
11
12 # Extract the JSON content from the LLM's response
13 body_json = api_response.json()
14 deepseek_output_text = body_json["choices"][0]["message"]["content"].strip()
15 entita = json.loads(deepseek_output_text)
16
17 # Save the final structured profile to the output S3 bucket
18 output_bucket = "file-output-profilo-onboarding"
19 output_key = file_name.replace(".json", "_deepseek.json")
20 s3.put_object(
21     Bucket=output_bucket,
22     Key=output_key,
23     Body=json.dumps(entita, ensure_ascii=False),
24     ContentType="application/json"
25 )

```

### 3.2.3 Profile Retrieval Function

The third Lambda function is responsible for providing the processed profile data to the frontend.

- **Trigger:** The function is activated by Amazon API Gateway, which receives a GET request from the client.
- **Responsibilities:** Its sole responsibility is to locate and return the most recent profile file.

This function does not receive a specific ID; instead, it implements logic to identify the latest file in the target bucket. To achieve this, it lists all objects in the bucket [13] and then, using a Python lambda function, sorts the list by the *LastModified* attribute to identify the most recent object.

```

1 BUCKET_NAME = "file-output-profilo-onboarding"
2
3 # --- inside the lambda_handler function ---
4
5 # 1. List all objects in the output bucket
6 resp = s3.list_objects_v2(Bucket=BUCKET_NAME)
7
8 if 'Contents' not in resp or len(resp['Contents']) == 0:
9     return {"statusCode": 404, ...} # Handle case where bucket is empty
10
11 # 2. Find the most recently modified object using a lambda function as the key
12 latest_obj = max(resp['Contents'], key=lambda x: x['LastModified'])
13 s3_key = latest_obj['Key']
14
15 # 3. Download that specific JSON file
16 obj_resp = s3.get_object(Bucket=BUCKET_NAME, Key=s3_key)
17 profile_obj = json.loads(obj_resp['Body'].read().decode('utf-8'))
18
19 # 4. Return the JSON content to the frontend
20 return {
21     "statusCode": 200,
22     "headers": { ... },
23     "body": json.dumps(profile_obj)
24 }

```

This strategy ensures that the frontend always receives the result of the latest onboarding process. The content of the JSON file is then read [14] and returned as the body of the HTTP response, ready to be displayed by the client application.



# Chapter 4

## Named Entity Recognition: A Comparison of Solutions

As highlighted in Chapter 2, the selection of a technology for Named Entity Recognition (NER) represents a critical step for the success of the project. The analysis of standard datasets has shown the poor adequacy of pre-trained models with respect to the specific domain of a dating application, characterized by colloquial language and by the need to recognize unconventional entities, such as personal interests and passions, character traits, preferences regarding the desired partner, and demographic or contextual information (age, city, profession).

This observation made the adoption of a "ready-to-use" solution impractical and requiring an in-depth comparative analysis. The aim of this chapter is to explore and critically evaluate three different paradigms for NER implementation, each with its own strengths and weaknesses:

1. **A managed NLP service (Managed Service):** represented by Amazon Comprehend, which offers pre-trained and customisable NER functionalities.
2. **A specialized Transformer model (Fine-tuning):** represented by a customized BERT model, which requires training on an ad-hoc created dataset.
3. **A Large Language Model (LLM) in Zero-Shot mode:** represented by DeepSeek, which leverages its generative capabilities to extract entities through natural language instructions (prompting).

Through the analysis of each approach, aspects such as ease of use, flexibility, costs, development time, and potential accuracy will be examined, in order to justify the final technological choice adopted for the prototype.

### 4.1 Approach 1: Amazon Comprehend

The first solution considered was Amazon Comprehend [17], a fully managed Natural Language Processing service provided by AWS. In theory, this option represented the fastest and most integrated path to add NER functionalities to the existing architecture.

#### 4.1.1 Advantages (Pros)

The main advantage of Amazon Comprehend lies in its nature as a managed service. This means that all the complexity related to infrastructure management, model training, and

maintenance is entirely handled by AWS. Although it was not adopted within the project, it is useful to highlight the main advantages that characterize Amazon Comprehend, in order to understand its potential and positioning compared to other solutions:

- **Immediate Integration:** Being a native service within the AWS ecosystem, its integration inside a Lambda function is extremely simple and requires only a few lines of code through the Boto3 SDK. This drastically reduces initial development time.
- **No Machine Learning Expertise Required:** To use its basic functionalities, no knowledge of machine learning is required. The pre-trained NER model can recognize a set of standard entities (such as PERSON, LOCATION, ORGANIZATION, DATE) quickly and with good accuracy on formal texts.
- **Reliability and Automatic Management:** As a managed service, Comprehend ensures that load distribution and availability are maintained without developer intervention.

#### 4.1.2 Limitations and Critical Issues (Cons)

A more in-depth analysis highlighted how Amazon Comprehend presents significant limitations that make it poorly suited to the specific needs of this project.

- **Inadequacy of Pre-trained Entities:** Comprehend's standard NER model has been trained on generic texts, such as news articles and web documents. As a result, the categories of recognized entities are limited and do not include those fundamental for a dating profile, such as personal interests and passions, profession, or partner preferences. A sentence like *"I like cinema and I am an altruistic person"* would be processed without recognizing *cinema* as an interest/hobby nor *altruistic* as a character trait, since these categories are not part of the service's predefined entity schema.
- **Rigidity, Complexity, and Costs of Customization:** To handle categories not covered by pre-trained entities, such as hobbies or character traits, Comprehend includes the Custom Entity Recognition function. However, the use of this functionality implies a complex and not very flexible process. Training a custom model requires the creation of a large annotated text dataset, a development requirement to which an economic impact is added, making the option impractical. The training process is a paid operation, but it is the usage model (inference) that is particularly problematic: to query the custom model, a dedicated endpoint must be created, billed on an hourly basis regardless of usage. Finally, once trained, the model proves rigid: for every modification, the entire costly annotation and retraining process must be repeated.

In conclusion, although Amazon Comprehend is an excellent solution for standard use cases, its lack of flexibility and the expensive customization requirements made it an impractical choice for this project, pushing the research towards more agile and adaptable solutions.

## 4.2 Approach 2: Fine-Tuning a Custom BERT Model

The second approach analysed involves the use of BERT, a language model already pre-trained on large corpora of general-purpose text, but not on specific tasks such as entity recognition. To make it suitable for the project's domain, an additional training phase (fine-tuning) was therefore required on an ad hoc annotated dataset.

Compared to a managed service such as Amazon Comprehend, which only provides a limited set of predefined entities, this solution offers the possibility of adapting the model to custom categories, more aligned with the needs of the voice-based onboarding system. The main advantage lies in greater flexibility and precision in entity recognition; the main

drawback, however, is the necessity of having a sufficiently large and representative annotated dataset in order to achieve good performance.

The following subsections will present the fundamental steps of this methodology, starting from the description of BERT's architecture, moving through the construction of the synthetic dataset and the preliminary data analysis, up to fine-tuning and evaluation, with the aim of assessing its effectiveness and real applicability within the context of the project.

### 4.2.1 The Transformer Architecture: BERT

BERT, which stands for Bidirectional Encoder Representations from Transformers, is a language model introduced by Google in 2018 [18] that revolutionized the field of Natural Language Processing (NLP). Its main innovation, which distinguished it from previous models (such as recurrent neural networks, RNNs, or the first autoregressive versions of GPT), consists in its ability to learn bidirectional text representations.

Unlike traditional models that process sequences unidirectionally — either left-to-right or right-to-left — BERT processes the entire sequence simultaneously. This allows the context of each word to be reconstructed by considering both the preceding and following terms, enabling a deeper understanding of linguistic relationships.

This approach is particularly effective in resolving semantic ambiguities: for instance, in a sentence like *"I'm looking for someone who loves reading yellow books"*, the term *yellow* could refer either to the colour or to a literary genre. BERT leverages context to assign the correct meaning.

The architecture of BERT is based exclusively on the **encoder** part of Transformers [19]:

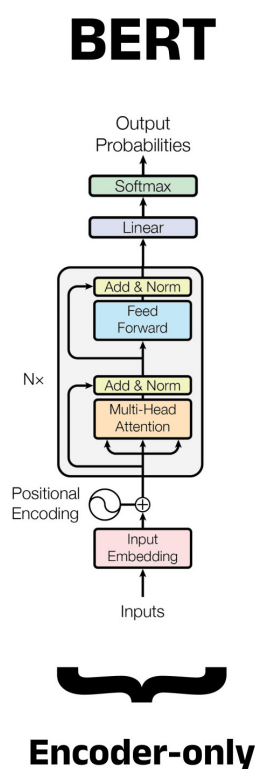


Figure 4.1: BERT Architecture Overview [20]

#### Input Embedding

Each token in the sequence is represented as the sum of three vectors:

- **Token embedding:** captures the lexical meaning of the word or subword.
- **Segment embedding:** distinguishes tokens belonging to different sentences (useful in tasks involving sentence pairs).
- **Positional embedding:** provides information about the token's position within the sequence, enabling the model to take word order into account.

#### Multi-Head Self-Attention Layer

The input embedding enters the Multi-Head Self-Attention mechanism, which in parallel computes the dependency relationships between each token and all the others in the sentence, also capturing long-range connections.

#### Feed-Forward Network and Normalization

Each encoder then applies a feed-forward neural network to every token, supported by residual connections and layer normalization, in order to stabilize learning and facilitate convergence.

#### Contextualized Output

After passing through the stack of encoders, the tokens obtain rich and contextualized representations. These are then processed by the final layer to generate classification probabilities, as in the case of entity recognition.

The success of BERT is based on a two-stage learning process:

### 1. Pre-training

In this stage, the model is trained on massive amounts of unlabeled text (the entire English Wikipedia and BookCorpus, approximately 3.3 billion words). The training relies on two main objectives:

- **Masked Language Model (MLM):** 15% of the input tokens are masked, and the model must predict them based on the bidirectional context. This forces BERT to build robust, context-sensitive representations.
- **Next Sentence Prediction (NSP):** the model receives two sentences as input and must determine whether the second one actually follows the first in the original text. This task supports the understanding of inter-sentence relationships and textual coherence.

### 2. Fine-tuning

After pre-training, BERT serves as a powerful general-purpose language model. However, to achieve strong performance on specific tasks (such as Named Entity Recognition, Question Answering, or Sentiment Analysis), it undergoes fine-tuning. In this phase, the entire model — not only the final layers — is retrained on smaller, task-specific datasets, adapting the learned weights to the requirements of the application. In the context of this project, fine-tuning was performed on the dataset built with Gemini for entity recognition in the voice domain.

BERT was released in two main versions: **BERT BASE** (12 layers, 768 hidden dimensions, 12 attention heads, 110 million parameters) and **BERT LARGE** (24 layers, 1024 hidden dimensions, 16 attention heads, 340 million parameters). Both have demonstrated unprecedented performance on standard benchmarks such as GLUE, SQuAD, and CoNLL-2003.

## 4.2.2 Dataset Creation via LLM

The manual creation of an annotated dataset, as discussed earlier, is a long and costly process. To overcome this obstacle, an innovative solution was adopted: the generation of a synthetic dataset using a Large Language Model (LLM), specifically Gemini, through its API [21].

The logic was based on providing the model with a structured prompt. This prompt not only asked for the generation of realistic sentences for a dating context but also included precise instructions to directly annotate the generated sentences according to the IOB encoding scheme. The goal was to produce a corpus focused on a rich and domain-specific set of entities, such as: NOME, ETA, CITTA, ALTEZZA, HOBBY, PROFESSIONE, PERS (personality traits), RANGEETA (desired age ranges), INTERESSE (desired partner characteristics), and GENERE. This process made it possible to create a training-ready text file containing approximately 2,800 annotated sentences.

This choice offered significant advantages, such as the rapid creation of hundreds of annotated examples — a result that would have required days of manual work — achieved at zero cost in terms of data generation. At the same time, some limitations emerged. The sentences tended to follow repetitive patterns (*"My name is X, I am Y years old, and I live in Z"*), reducing the naturalness and variety typical of real spoken language. Moreover, the dataset reflected the biases of the model used, with the risk that BERT, once trained, would perform very effectively on sentences similar to those artificially generated but prove less robust when applied to real and more heterogeneous examples.

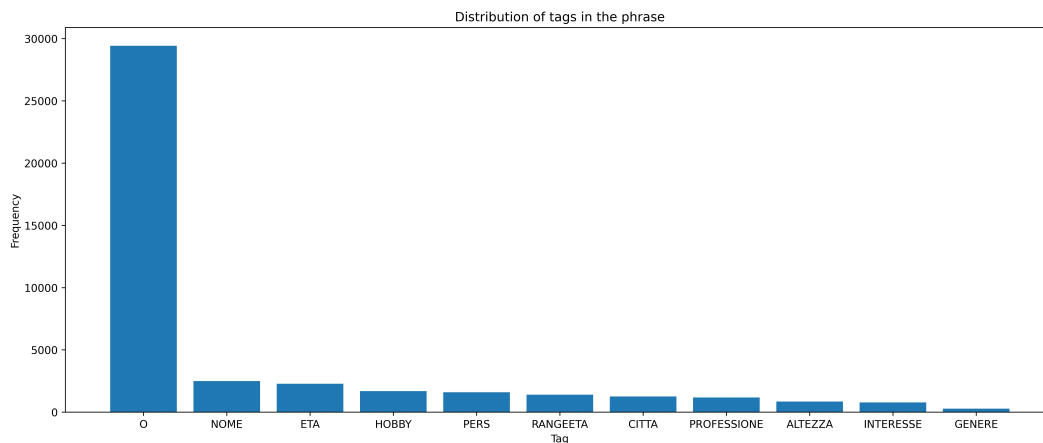
### 4.2.3 Exploratory Analysis and Model Training

After generating the synthetic dataset, the next step focused on training a BERT model for the specific task of Named Entity Recognition. This process was divided into two main phases: a preliminary Exploratory Data Analysis (EDA) of the dataset, followed by fine-tuning of the transformer model.

#### Exploratory Data Analysis (EDA)

Before proceeding with training, a detailed analysis of the dataset generated by Gemini was conducted. This phase, implemented in the initial part of the development notebook, was crucial for understanding the characteristics of the corpus and preparing it in the format required by the model. The main operations included reading the text file, converting the annotated sentences into a structured format (via a Pandas DataFrame), mapping IOB labels to numerical values, and producing descriptive statistics on the dataset. In addition to these basic checks, key aspects such as average sentence length, token distribution, and most importantly the frequency of the different entities were analysed.

One of the most important aspects examined was the distribution of labels, to ensure a sufficient number of examples for each entity of interest. The following figure, generated during the analysis, shows the frequency of each tag in the training corpus.



**Figure 4.2:** Distribution of labels (tags) in the training dataset. The "O" (Outside) tag is clearly dominant, as it is typical in NER tasks; the annotated entities include NOME, ETA, CITTA, PROFESSIONE, HOBBY, PERS, RANGEETA, ALTEZZA, INTERESSE, and GENERE.

This analysis confirmed that, although unbalanced toward the "O" tag (an intrinsic characteristic of all NER datasets), the corpus contained an adequate number of examples for all custom entities, validating the quality of the synthetic generation.

#### BERT Model Fine-Tuning

For the fine-tuning process, a pre-trained model specifically designed for the Italian language was selected. In line with the previous section, the chosen model was **BERT BASE**, specifically dbmdz/bert-base-italian-cased [22]. This model, released by the DBMDZ group, is a benchmark for Italian text processing and has been trained on a large corpus of Italian data, representing an excellent starting point for specialisation on our task. Fine-tuning was carried out using the Hugging Face Transformers library [23], which provides high-level tools for training language models. The implementation logic focused on configuring the model for the `TokenClassification` task (i.e., NER) and defining the training arguments.

The following code snippet shows the key configuration of the training process:

```

1 from transformers import BertForTokenClassification, TrainingArguments, Trainer
2
3 # Load the pre-trained BERT BASE model for token classification,
4 # specifying the number of unique labels from our dataset.
5 model = BertForTokenClassification.from_pretrained(
6     "dbmdz/bert-base-italian-cased",
7     num_labels=len(label_list) # label_list contains all unique tags (B-NOME, I-NOME, etc.)
8 )
9
10 # Define the training arguments for the fine-tuning process.
11 training_args = TrainingArguments(
12     output_dir="./results",
13     evaluation_strategy="epoch", # Evaluate performance at the end of each epoch
14     learning_rate=2e-5,
15     per_device_train_batch_size=16,
16     per_device_eval_batch_size=16,
17     num_train_epochs=3,
18     weight_decay=0.01,
19 )
20
21 # Initialize the Trainer, which handles the training loop.
22 trainer = Trainer(
23     model=model,
24     args=training_args,
25     train_dataset=tokenized_datasets["train"],
26     eval_dataset=tokenized_datasets["test"],
27     data_collator=data_collator,
28     tokenizer=tokenizer,
29     compute_metrics=compute_metrics # A function to calculate Precision, Recall, F1
30 )
31
32 # Start the fine-tuning process
33 trainer.train()

```

As shown in the code, the `BertForTokenClassification` model is loaded and configured with the exact number of labels present in the dataset. The `TrainingArguments` define the hyperparameters of the process, such as learning rate, batch size, and number of epochs (complete training cycles). The Hugging Face `Trainer` abstracts away the complexity of the training loop, handling model optimization and periodic evaluation of performance.

Training was monitored through the metrics of Precision, Recall, and F1-Score, achieving an **average F1 of 0.80**, a result that confirmed the strong theoretical effectiveness of this approach.

#### 4.2.4 Advantages and Limitations of the BERT Approach: from Fine-Tuning to Prompt Engineering

##### Advantages (Pros):

- **High Accuracy:** Fine-tuning a model like BERT is one of the most effective techniques for NER. Once specialized on our dataset, the model learns to recognize custom entities with great accuracy, leading to potentially very reliable results.
- **Autonomy and Speed:** A trained BERT model is a self-contained file that can be executed directly within the system without relying on external connections or third-party services. This ensures immediate response times (low latency) and greater control over the data, with significant benefits also in terms of privacy.

##### Limitations in the Context of the Project (Cons):

- **Model Size and Deployment Costs:** Even in its "BASE" version, a trained BERT model exceeds 1 GB in size, compared to the 250 MB limit imposed by AWS Lambda. The alternative of using SageMaker was discarded because it was too costly: the service requires a 24/7 active endpoint, with high fixed costs and additional expenses if a GPU is needed to guarantee adequate performance.

- **Long-Term Management Complexity:** A fine-tuned model is not a "plug-and-play" solution; it requires updates, periodic checks, and, if necessary, new training cycles. This introduces additional operational overhead that would have made long-term maintenance more complex.

This experience, however, raised a fundamental question: if a Large Language Model (Gemini) was powerful enough to generate an entire high-quality annotated dataset, why face the complexity of using that dataset to train a second model (BERT)? Wouldn't it be more straightforward and efficient to use an LLM directly to perform the NER task in real time?

This question opened the way for exploring the third approach: the use of an LLM through prompt engineering, which will be discussed in the following section.

### 4.3 Approach 3: Zero-Shot NER with Large Language Models

The third and final approach analysed and adopted in the project differs from the previous ones. Instead of using a preconfigured service (such as Amazon Comprehend) or training a specialized model (such as BERT), this solution leverages the ability of a general-purpose Large Language Model (LLM) to tackle complex tasks in zero-shot mode. In other words, the model does not require additional training phases, but is guided exclusively through natural language instructions.

As already mentioned in the description of the backend (Chapter 3.2.2), the approach is based on so-called prompt engineering: the definition of a carefully designed text that instructs the LLM to extract the desired entities and return them in a structured JSON format. Specifically, the OpenRouter service was used [15], a platform that aggregates different language models and provides a unified API interface.

In this context, the "model" is no longer the result of a fine-tuning process but rather the prompt itself, which becomes the crucial element to guide the system's behaviour.

From a technological perspective, the implementation was carried out through simple HTTP calls, managed with Python's standard requests library [16], thus ensuring lightweight integration compatible with AWS Lambda's serverless environment.

The final choice fell on DeepSeek, not only for economic convenience but above all for its ability to provide stable and immediately usable API access within a prototyping context. Unlike alternative solutions such as ChatGPT, available only through subscription, DeepSeek allowed rapid model integration without additional costs, making it the most concrete and effective solution for the project's objectives. It is important to note, however, that the approach remains independent of the specific model: preliminary tests conducted with Gemini also confirmed that comparable-quality results could have been obtained with other LLMs.

#### 4.3.1 Advantages (Pros)

This approach proved successful for the project, offering a combination of flexibility and implementation simplicity that the other solutions could not provide:

- **Flexibility and Development Speed:** This is the most significant advantage. To add a new entity or modify an existing one, there is no need to retrain a model or annotate a dataset. It is sufficient to update the prompt string inside the Lambda function. This allows for iteration and modification of the extraction logic in just a few seconds, providing great agility in development.
- **Compatibility with Serverless Architectures:** Unlike BERT, this approach does not require loading a heavy model. The Lambda function remains extremely lightweight

(a few MB), since it only contains the code to query an API, making it consistent with the characteristics and constraints of the serverless environment.

- **Absence of Fixed Costs:** The adoption of DeepSeek did not incur any usage costs during project development. This made the approach not only technically effective but also economically advantageous, allowing for rapid testing and validation of the prototype without any financial investment.
- **High Context Understanding Capability:** Modern LLMs, having been trained on vast amounts of text, possess a remarkable ability to understand natural language, including colloquial expressions, complex sentences, and contextual nuances, making them potentially very robust.

### 4.3.2 Limitations and Critical Issues (Cons)

Despite the clear advantages, the LLM-based approach also introduces new considerations to keep in mind in a production context:

- **Dependence on External APIs and Variability:** The execution of NER entirely relies on API calls to a model hosted by third parties. This has two main consequences: on the one hand, response latency can vary depending on connection and service availability; on the other, the overall system reliability remains tied to the chosen provider.
- **Privacy and Data Security Considerations:** Even though there are no direct costs associated with using the selected model, privacy remains an issue. Sending potentially sensitive information to an external service requires careful evaluation of data handling and storage policies, especially in a sensitive domain such as dating.

Despite these considerations, for the objectives of this project — namely the rapid prototyping of an innovative, flexible, and low-cost system — the advantages offered by this approach proved not only superior but decisive, making it by far the most pragmatic and effective solution among those analysed.

## 4.4 Critical Comparison: Fine-Tuned BERT vs. Zero-Shot LLM

After analysing the three approaches to Named Entity Recognition, the focus now shifts to the direct comparison between the two most relevant solutions: the fine-tuning of a BERT model and the use of an LLM in zero-shot mode. Both strategies demonstrated significant strengths but also structural limitations. This section systematically compares the two options, highlighting how the greater flexibility, simplicity, and contextual understanding capability of LLMs made this approach the most suitable for the project's objectives.

The following table summarizes the main comparison criteria between the two approaches:

| Criterion                 | Approach 2: Fine-Tuning BERT                                                                                                                          | Approach 3: Zero-Shot LLM (DeepSeek)                                                                                                               |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Accuracy                  | High. Once trained on a specific dataset, the model becomes domain-expert, achieving high F1-scores (0.80 in testing).                                | Very High. Modern LLMs has remarkable contextual and colloquial language comprehension, but the output is not guaranteed to be 100% deterministic. |
| Flexibility and Iteration | Low. Adding or modifying an entity requires re-annotation of the dataset and a new, costly training cycle.                                            | High. A modification only requires updating a text string (the prompt), enabling iterations in seconds.                                            |
| Time and Complexity       | High. Using BERT requires building and annotating a specific dataset, executing intensive training, and managing a complex deployment infrastructure. | Low. No training required: it is sufficient to define the prompt and integrate a simple API call, with rapid and lightweight development.          |
| Operational Costs         | High and Fixed. Deploying a custom model requires a dedicated endpoint (e.g., SageMaker), with hourly costs independent of actual usage.              | None. In the case of the chosen model (DeepSeek), API access was free, eliminating execution costs entirely.                                       |
| Serverless Compatibility  | Incompatible. The model size (over 1 GB) largely exceeds AWS Lambda's limits (250 MB).                                                                | Perfectly Compatible. The Lambda function remains lightweight, containing only the logic to query an external API.                                 |
| Latency                   | Low and Stable. Being a self-hosted model, responses are immediate and do not depend on external services.                                            | Variable. Latency depends on the API provider's response time and network connection.                                                              |

Table 4.1: Comparison of NER approaches: Fine-Tuning vs. Zero-Shot LLM.

#### 4.4.1 Testing on Real-World Cases

To concretely evaluate the differences between the two approaches, it was useful to analyse how they perform with sentences drawn from realistic scenarios. While in simple situations both models manage to correctly extract entities, it is with increasing natural language complexity that BERT's limitations emerge and, conversely, the LLM's demonstrate their ability to handle context in a more flexible and intelligent manner.

##### Simple Sentence

This is the ideal scenario for a model like BERT, trained on a dataset with a similar structure.

*"Ciao, sono Giulia, ho 29 anni e sono alta un metro e settanta. Come professione faccio l'avvocato. Mi definisco una persona molto altruista e vulnerabile. Nel tempo libero adoro leggere e fare trekking. Cerco un uomo tra i 30 e i 35 anni."*

**BERT Output:**

```

1 {
2   "NOME": "Giulia,",
3   "ETA": "29",
4   "ALTEZZA": "un metro e settanta.",
5   "PROFESSIONE": "1' avvocato",
6   "PERS": "molto altruista e vulnerabile.",
7   "HOBBY": ["leggere", "trekking."],
8   "INTERESSE": "uomo",
9   "RANGEETA": "30 e i 35"
10 }

```

BERT correctly recognizes most entities. However, the output, although structured, reveals its nature as a token classifier: it includes punctuation ("Giulia,"), shows tokenization artifacts ("1 ' avvocato"), and fails to logically group complex entities.

**LLM Output:**

```

1 {
2   "NOME": "Giulia",
3   "ETA": "29",
4   "GENERE": "Donna",
5   "PERS": ["Altruista", "Vulnerabile"],
6   "PROFESSIONE": "Avvocato",
7   "ALTEZZA": "1.70",
8   "HOBBY": ["leggere", "trekking"],
9   "INTERESSE": "uomo",
10  "RANGEETA": {"min": "30", "max": "35"}
11 }

```

The LLM not only extracts but also interprets and normalizes the information. It returns a clean JSON, derives gender from context, normalizes height into numeric format, structures the age range, and logically groups personality traits and hobbies into lists.

**Complex Sentences**

The analysis of more complex and realistic sentences highlights the difficulties encountered by BERT and, at the same time, the greater robustness of LLMs in contextual interpretation.

**1. Temporal Context (Past Profession)**

*"Mi chiamo Marco, ho 32 anni e vivo a Bologna. Lavoro come insegnante di storia in un liceo, ma in passato ho fatto l'archeologo in Grecia per alcuni anni. ..."*

**BERT Output:**

```

1 {
2   "NOME": "Marco",
3   "ETA": "32",
4   "CITTA": "Bologna.",
5   "PROFESSIONE": ["insegnante", "storia", "archeologo"],
6   ...
7 }

```

**BERT Limitation:** The model does not understand temporal context. It identifies "insegnante", "storia" and "archeologo" as professions, treating them all as current, producing incorrect and unusable information.

**LLM Output:**

```

1 {
2   "NOME": "Marco",
3   "ETA": "32", "GENERE": "Uomo",
4   "CITTA": "Bologna",
5   "PROFESSIONE": "insegnante di storia",
6   ...
7 }

```

**LLM Superiority:** The LLM understands the phrase *"ma in passato ho fatto..."* and correctly discards "archeologo", extracting only the current profession and aggregating "insegnante di storia".

## 2. Handling Negation

*"... Sono una persona istintiva e flessibile ma non altruista, mi piace cucinare e suonare la chitarra. Cerco una donna tra i 30 e i 34 anni."*

### BERT Output:

```
1 {  
2 ...  
3 "PERS": "istintiva e flessibile ma non altruista",  
4 "HOBBY": ["cucinare", "suonare", "chitarra."],  
5 "INTERESSE": "donna",  
6 "RANGEETA": "30 e i 34"  
7 }
```

BERT Limitation: This represents the most critical limitation. BERT labels the entire string, including the negation "non altruista", as a single personality trait. The system would interpret a meaning opposite to that intended by the user.

### LLM Output:

```
1 {  
2 ...  
3 "PERS": ["Flessibile", "IstintivÉ"],  
4 "HOBBY": ["cucinare", "suonare la chitarra"],  
5 "INTERESSE": "donna",  
6 "RANGEETA": {"min": "30", "max": "34"}  
7 }
```

LLM Superiority: The LLM demonstrates real semantic understanding. It processes the negation and correctly excludes "altruista" from the list of personality traits, interpreting the actual meaning of the sentence.

Although the BERT-based approach (Approach 2: Fine-Tuning a Custom BERT Model) guarantees high accuracy on simple sentences, the examples clearly show its fragility when faced with the complexities of real language. Its inability to correctly handle temporal context and negations makes it unreliable for an application that must faithfully interpret user descriptions.

Consequently, the choice fell on Approach 3: Zero-Shot NER with Large Language Models

This solution proved to be not only more pragmatic from an architectural and cost perspective, but above all significantly superior in terms of intelligence and robustness. The LLM's ability to understand context, interpret nuances and negations, and return structured output represents the fundamental added value that justified its adoption. For a project aiming to create natural and reliable interaction, the flexibility and deep language understanding offered by the LLM proved to be an essential requirement, making it the most technologically advanced and effective choice among those analysed.



# Chapter 5

## Conclusions and future developments

The thesis pursued the goal of designing and developing an innovative prototype for user onboarding, based on voice interaction and implemented using a serverless architecture. Through comparative analysis of different Named Entity Recognition technologies, it was possible to identify a pragmatic and effective solution. The final decision to adopt a Large Language Model (LLM) in zero-shot mode made it possible to overcome the rigidity and costs of managed services, as well as the implementation barriers posed by specialized models such as BERT. This final chapter aims to review the challenges encountered and then explore in more depth possible future improvements to the prototype and its potential applications in other market contexts.

### 5.1 Challenges Encountered and Adopted Solutions

During the research and development process, several challenges emerged, both conceptual and technical. Resolving these challenges was an integral part of the thesis work.

- **Lack of Domain-Specific Data:** The first obstacle was the absence of public datasets for NER in a dating context. The solution adopted was to **generate a synthetic dataset using an LLM (Gemini)**. While this approach has limitations — such as potential lack of diversity — it has proven to be an effective strategy for quickly overcoming the data collection barrier.
- **BERT Model Incompatibility:** The most significant obstacle was of a technical nature. Although fine-tuning demonstrated the theoretical effectiveness of BERT, **the model's size (>1 GB)** exceeded the deployment limits of AWS Lambda. Evaluating Amazon SageMaker as an alternative highlighted a fixed, hourly cost model incompatible with the project's objectives. This led to discarding the solution, showing the importance of assessing not only model accuracy but also its practical and operational sustainability. The impracticality and economic unsustainability of this path ultimately made it necessary to explore an alternative paradigm, leading to the adoption of an **LLM in zero-shot mode**.
- **Variability of LLM Output:** Using a generative model in zero-shot mode, such as DeepSeek, introduced the possibility of non-deterministic outputs — i.e., responses not always identical given the same input. In some cases, the model returned slight formatting variations or incomplete JSON structures. To minimise these effects, two complementary strategies were adopted: on the one hand, **careful prompt design**, so as to constrain the model to return only responses in JSON format; on the other

hand, the integration of **validation and error handling logic** (try-except) within the Lambda function, so as to intercept and correct any anomalies in the generated content.

## 5.2 Improvements and Future Developments

The current prototype also represents a solid basis for numerous future developments that could transform it into a fully operational and more sophisticated application.

- **Conversational Onboarding:** The most natural evolution of the system would be to move beyond a single voice message and create an interactive dialogue. Instead of a user monologue, a virtual assistant could ask targeted questions (*"Tell me a bit about yourself. What job do you do?"*, *"What do you like to do in your free time?"*). Conversation state management (for example via Amazon DynamoDB) would enable this approach, allowing for more accurate data collection, reduced ambiguities, and a more natural, engaging user experience.
- **Emotional and Prosodic Analysis:** A significant enrichment of the user profile could derive from analysing non-verbal aspects of communication. Going beyond simple transcription, sentiment analysis models could be integrated to determine the emotional tone of speech, and prosodic analysis algorithms could evaluate rhythm, pauses, and intonation. Such data would provide insights into personality (e.g., calm, energetic, reflective), adding a level of depth that text alone cannot capture.
- **Dynamic Multilingual Support:** To make the application global, multilingual support is essential. The current architecture can be extended by integrating Amazon Transcribe with automatic language identification as the first step in the flow. The NER Lambda function could then dynamically adapt by selecting the appropriate prompt and instructions for the detected language, enabling the system to transparently handle users worldwide.
- **Integration with the Platform's Serverless Database:** While the current storage of profiles in S3 is suitable for a prototype, a real application requires integration with a scalable and performant database, such as Amazon DynamoDB, already adopted in the Hindeep platform. This would allow native, scalable management of profiles, ensuring not only secure persistence of data but also real-time complex queries (e.g., *"find all users in Milan interested in trekking"*), essential for matching functionality.

## 5.3 Applications in Other Contexts

The flexibility of the LLM-based serverless architecture makes it easily adaptable to many other market contexts, converting any voice-based data collection process into a structured workflow.

- **Human Resources and Automated Recruiting:** The system could transform the initial steps of candidate selection. Applicants could record a short self-presentation or answer pre-screening questions. The system would extract information such as work experience, technical skills, language proficiency, and years of experience, automatically generating a preliminary candidate profile to support recruiters.
- **Customer Support and Automated Ticketing:** In customer service, users could leave a voice message describing a problem. The system would automatically extract information such as customer name, order number, product type, and a brief summary of the issue. These details would be used to generate a pre-filled support ticket, reducing processing times and improving service efficiency.

- **Medical Dictation and Electronic Health Records:** In healthcare, a doctor might dictate notes from a medical visit. The system could extract reported symptoms, hypothesized diagnosis, prescribed medicine, dosage, and follow-up dates. This would automate the compilation of electronic medical records, significantly reducing the doctor's administrative workload, allowing more time for patient care, while reducing the risk of transcription errors.
- **Legal and Insurance Automation:** An insurance adjuster could verbally describe a car accident. The system could extract details such as accident date, location, vehicles involved, damage type, and witness information, automating the compilation of the first part of the claim and accelerating the entire settlement process.

## 5.4 Final Considerations

In conclusion, this thesis has explored the design and development of a voice-based onboarding system, starting from a concrete market need and navigating the challenges and opportunities of modern cloud and artificial intelligence technologies. The comparative analysis of different Named Entity Recognition approaches demonstrated that, beyond the theoretical effectiveness of specialized models, true innovation today lies in the ability to adopt pragmatic, flexible, and economically sustainable solutions.

The adoption of a serverless architecture, combined with the power of a Large Language Model in zero-shot mode, not only overcame technical and cost barriers but also validated a new development paradigm: more agile, faster, and more accessible. The prototype created is therefore not just a solution to a specific problem, but also a concrete demonstration of how it is possible, today, to build complex AI applications at low cost, paving the way for a future in which the interaction between humans and machines will be increasingly natural, intuitive, and, above all, human.

A special acknowledgment goes to Hindeep, whose support made this work possible and whose vision may transform these solutions into tools capable of fostering authentic connections between people, accompanying them on an increasingly meaningful and fitting path of encounter.



# References

- [1] *AWS Certified Solutions Architect Official Study Guide: Associate Exam*. Sybex, 2016. (Cited on pages 9, 10, and 14)
- [2] Amazon Web Services, “Amazon web services official website.” <https://aws.amazon.com>. Accessed: 2025-09-16. (Cited on pages 10 and 11)
- [3] T. Felke-Morris, *Web Development and Design Foundations with HTML5*. Pearson, 9 ed., 2019. (Cited on pages 11 and 12)
- [4] J. Li, A. Sun, J. Han, and C. Li, “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, 2022. (Cited on page 12)
- [5] M. Viviani, “Web and social media search and analysis – lecture slides,” 2025. Course material, University of Pavia. (Cited on page 13)
- [6] D. Jurafsky and J. H. Martin, *Speech and Language Processing (3rd ed. draft)*. 2023. (Cited on pages 13 and 14)
- [7] MDN Web Docs, “Mediarecorder api.” [https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder\\_API](https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder_API). (Cited on page 18)
- [8] MDN Web Docs, “Fetch api.” [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API). (Cited on pages 18, 19, and 20)
- [9] MDN Web Docs, “Filereader api.” <https://developer.mozilla.org/en-US/docs/Web/API/FileReader>. (Cited on pages 18 and 19)
- [10] Chart.js Contributors, “Chart.js documentation.” <https://www.chartjs.org/docs/latest/>. (Cited on pages 18 and 20)
- [11] Python Software Foundation, “base64 - base16, base32, base64, base85 data encodings.” <https://docs.python.org/3/library/base64.html>. (Cited on page 21)
- [12] Python Software Foundation, “uuid - uuid objects according to rfc 4122.” <https://docs.python.org/3/library/uuid.html>. (Cited on page 21)
- [13] Amazon Web Services, “Boto3 documentation.” <https://boto3.amazonaws.com>. (Cited on pages 21 and 23)
- [14] Python Software Foundation, “json - json encoder and decoder.” <https://docs.python.org/3/library/json.html>. (Cited on pages 22 and 23)
- [15] OpenRouter, “Openrouter: Unified api for large language models.” <https://openrouter.ai>. Accessed: 2025-09-17. (Cited on pages 22 and 31)

- [16] Requests Contributors, "Requests: HTTP for humans." <https://requests.readthedocs.io/en/latest/>. (Cited on pages 22 and 31)
- [17] Amazon Web Services, "Amazon comprehend." <https://aws.amazon.com/comprehend>. (Cited on page 25)
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018. (Cited on page 27)
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. (Cited on page 27)
- [20] V. Singh, "Llm architectures explained: Bert (part-8)," 2024. (Cited on page 27)
- [21] Google, "Gemini api documentation." <https://ai.google.dev/gemini-api>. (Cited on page 28)
- [22] DBMDZ, "bert-base-italian-cased (hugging face model card)." <https://huggingface.co/dbmdz/bert-base-italian-cased>. (Cited on page 29)
- [23] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, *et al.*, "Transformers: State-of-the-art natural language processing," in *EMNLP 2020: System Demonstrations*, 2020. (Cited on page 29)